

PowerMCP 实践指南

让 LLM 真正操作电力仿真软件

仓库: Power-Agent/PowerMCP | **许可:** MIT | **代码:** 约 26,000 行 Python | **当前版本:** powermcp 0.4.0 (Alpha)

定位: 电力系统软件的 MCP 服务器集合，把 PSS/E、PowerWorld、OpenDSS、pandapower 等 **16 款** 软件的 API 封装成 LLM 可直接调用的工具

适用读者: 电网工程师、新能源开发商技术团队、电力 AI 应用开发者

一、这个项目到底解决什么问题

1.1 电力 AI 最硬的那道墙

假设你让一个通用大模型回答："把 IEEE 39 节点系统的 L7 线路断开后，哪条线路会过载？"

它可能会给你一个听起来很专业的答案。但你无法验证它是真的算过，还是编的。

原因很直接：电力行业的核心仿真工具（PSS/E、PowerWorld、PowerFactory、PSCAD、PSLF）几乎全部是闭源商业软件，LLM 无法访问、无法调用、无法验证。模型再聪明，也只能停留在"文本层面"，无法触及"物理真实"。

PowerMCP 的解法很朴素但很彻底：**为每一款电力软件写一个 MCP 服务器，把软件的底层 API 变成 LLM 可以直接调用的工具。**

用户提问

↓

LLM (Claude / GPT / 本地模型)

↓ MCP 协议

PowerMCP 服务器 (16 个)

↓ 调用厂商 API

PSS/E · PowerWorld · OpenDSS · pandapower · PyPSA · ANDES ...

↓ 返回真实计算结果

LLM 基于物理结果作答

← 而不是基于想象

MCP 是什么: Anthropic 提出的 Model Context Protocol (模型上下文协议)，作用是给 AI 应用提供统一的工具连接标准。你可以把它理解为"AI 世界的 USB-C 接口"——不论什么模型、什么软件，都通过同一个协议对接。

1.2 一句话总结价值

PowerMCP 的价值不在于让 LLM 更聪明，而在于让 LLM 的结论第一次变得"可验证"。

二、工具矩阵：16 款软件怎么选

这是实践中最常被问到的问题。先看清楚全景，再决定装什么。

2.1 完整清单

工具	类型	软件领域	主要用途	安装方式
pandapower	开源	稳态 AC 分析	潮流、N-1 分析、网络建模	核心内置
PyPSA	开源	规划与运行优化	OPF、投资优化、容量扩张	核心内置
PowerIO	开源	格式转换	跨软件案例翻译、矩阵构建	核心内置
OpenDSS	开源	配电网分析	馈线潮流、日能量、谐波	[opendss]
ANDES	开源	动态安全/小信号	潮流、特征值、时域仿真	[andes]
Egret	开源	市场与运行优化	DC-OPF、AC-OPF、机组组合	[egret]
surge	开源	输电分析全流程	PTDF/LODF、OPF、ATC、SCED/SCUC	[surge]
LTSpice	免费	电路仿真	网表生成、无头仿真、波形	[ltspice]
GenX	开源 (Julia)	容量规划	容量结果聚合、分区绘图	[genx]
HOPE	开源 (Julia)	电力系统优化	本地/远程双入口	[hope]
PowerWorld	商业	输电网稳态分析	潮流、N-1、PTDF/LODF、Jacobian	[powerworld]
PSS/E	商业	输电网潮流与动态	潮流求解 + 全量 PSSPY 命令桥接	[psse] + 路径配置
PSLF	商业	GE 潮流与稳定	拓扑编辑、N-1 分析	[pslf] + 路径配置
PowerFactory	商业	机电暂态/RMS	RMS 仿真、潮流、短路、结果导出	[powerfactory] + 路径配置
PSCAD	商业	电磁暂态仿真	项目管理、仿真自动化	[pscad-windows]
PLEXOSDB	商业 (免许可)	电力市场建模	29 个 CRUD 工具 + Sienna 转换	[plexosdb] (需手动装)

本表共 16 行。其中 PowerIO 由依赖包提供（本仓不含其源码），其余 15 个 server 位于本仓各自目录下，可独立运行。

2.2 选型建议（按团队类型）

你的情况	推荐组合	理由
初次试水 / 教学演示	pandapower + PyPSA + PowerIO	零成本、零配置、pip 即装即用
高校科研	上述 + surge + ANDES + Egret	覆盖输电全流程 + 动态分析 + 优化
新能源并网研究	pandapower + OpenDSS + PyPSA	配电网 + 稳态 + 规划
电网公司调度侧	powerworld 或 psse + PowerIO	匹配现有生产工具链
电力市场分析	surge + Egret + PLEXOSDB	市场出清与调度
电磁暂态研究	pscad + PowerFactory + LTSpice	需要商业许可与 Windows

务实提醒：如果你还不确定要不要投入商业软件许可，先用纯开源组合跑通一个完整案例。开源的 pandapower + PyPSA + surge 已经能覆盖相当大比例的日常分析需求，验证价值后再决定是否扩充。

三、安装与配置（实操）

3.1 最简安装路径

```
# 1) 安装核心（含 pandapower + PyPSA + PowerIO, 无需任何配置）
pip install powermcp

# 2) 运行交互式向导
powermcp install
```

向导会依次完成三件事：

- 1. 让你勾选工具（↑ ↓ 移动，空格键切换选中，回车确认）
- 2. 采集闭源软件的本地安装路径（如 PSS/E、PSLF 的 Python 库路径）
- 3. 自动写入 MCP 客户端配置（支持 Claude Desktop、Claude Code、Codex CLI 三类）

注意：在勾选界面直接按回车，只会保留预选项（pandapower + PyPSA + PowerIO）。一定要用空格来增减选择。

3.2 命令行直选（不想用勾选界面）

```
powermcp install --tools psse,andes      # 核心 + 指定工具
powermcp install --all                   # 当前平台所有可用工具
powermcp install --dry-run               # 只预览改动，不实际写入
powermcp install --yes                   # 非交互，只装核心
```

一个重要特性：重新运行 `powermcp install` 时，上次已安装或已配置的工具会被自动预勾选，所以它是"更新"而不是"重置"。这对逐步扩充工具集的团队很友好。

3.3 安装衍生工具包（可选）

```
pip install powermcp[psse]           # PSS/E 支持
pip install powermcp[andes,openss]   # 一次装多个
pip install powermcp[opensource]     # 全部开源工具
pip install powermcp[all]            # 全部（闭源软件仍需本地安装）
```

实测提醒： `pip install powermcp[opensource]` 会一次带入 7 个开源 extra，其中 `egret` 会额外拉入 `pyomo`（较重）、`surge` 是 `pyo3/maturin` 轮子（仅支持 Python 3.12–3.14）。若只想要其中几个，用逗号直选更省事。另有一个反直觉的实测结论：`[genx] extra` **不增加任何新包**——它声明的 `matplotlib` / `pandas` 已由核心的 `pandapower` 传递带入。

3.4 闭源工具的路径配置

这些工具包装的是本地安装的商业软件，PowerMCP 需要知道它们在哪。配置存储在 `~/.powermcp/config.toml`。

工具	配置键	示例
PSS/E	<code>psse.python_lib</code> 、 <code>psse.bin</code>	<code>powermcp config set psse.python_lib "C:\Program Files\PTI\PSSE36\36.2\PSSPY311"</code>
PSLF	<code>pslf.python_lib</code>	<code>powermcp config set pslf.python_lib "C:\Program Files\GE PSLF\PSLF_PYTHON"</code>
PowerFactory	<code>powerfactory.python_path</code>	<code>powermcp config set powerfactory.python_path "...\\DIgSILENT\\PowerFactory 2024\\Python\\3.11"</code>
LTSpice	<code>ltspice.exe</code> （自动探测）	通常在标准位置自动找到，无需手动配置
HOPE	<code>hope.repo_root</code> 、 <code>hope.julia_bin</code>	<code>powermcp config set hope.repo_root "C:\src\HOPE"</code>
GenX	<code>genx.repo_root</code>	<code>powermcp config set genx.repo_root "/home/me/GenX.jl"</code>
PowerWorld	无需配置	<code>esa</code> 通过 COM 自动发现正在运行且有许可的 Simulator
PSCAD	无需配置	由 <code>pip install powermcp[pscad-windows]</code> 提供 <code>mhi-pscad</code>

3.5 验证安装

```
powermcp doctor      # 逐个检查工具的依赖与路径配置
powermcp list        # 列出可用工具、extras、平台限制
```

`doctor` 是排查问题的第一站。如果某个工具报错，先看它，再决定是装依赖还是配路径。

doctor 报 ok 不等于工具可用： `doctor` 只检查两件事——依赖包能否导入、配置路径是否存在。它**不验证工具真的能被调用**。本指南第六章给出的可用性结论，是在 `doctor` 之外另外做了运行时核验（实际拉起 server、调 `list_tools` 取真实返回值）得到的。

3.6 生成的客户端配置长什么样

```
{
  "mcpServers": {
    "powermcp_pandapower": { "command": "powermcp", "args": ["run", "pandapower"] },
    "powermcp_pypsa":      { "command": "powermcp", "args": ["run", "pypsa"] },
    "powermcp_powerio":    { "command": "powermcp", "args": ["run", "powerio"] },
    "powermcp_psse":       { "command": "powermcp", "args": ["run", "psse"] }
  }
}
```

工作区陷阱（值得单独提醒）：有用户报告 Windows 上的 Codex Desktop 应用会覆盖 `~/.codex/config.toml`。如果你同时使用 Desktop 和 CLI，Desktop 改过配置后需要重新运行一次 `powermcp install`。Codex CLI 本身不受影响。

3.7 不用安装，直接从克隆运行

每个服务器都是独立脚本，克隆仓库后可直接跑：

```
python pandapower/panda_mcp.py
python PSSE/psse_mcp.py          # 会读取 ~/.powermcp/config.toml
python -m powerio.mcp            # powerio 自带服务器
```

推荐做法（本指南的实测环境即如此）：用一个**隔离的虚拟环境**安装，而不是装在系统 Python 里。实测做法是 `uv venv .venv --python 3.12` 后 `uv pip install -e "[hope,surge]"`，全程未触碰任何 conda 环境或系统配置。原因是 PowerMCP 的依赖有版本上界（如 `pypsa>=0.35.2,<2`、`pandas>=2.0,<3`），与既有环境混装容易冲突。

四、各工具能力速查

本章按 server 逐个列出工具清单。工具数为仓库静态统计 + 运行时核验的结果；标注 ★ 的 server 已做过运行时 `list_tools` 核验。

4.1 PowerWorld（输电网稳态分析）

最完整的输电网稳态工具集，共 14 个工具：

工具	作用
<code>open_case(case_path)</code>	打开案例文件，返回基本信息
<code>run_powerflow(solution_method='RECTNEWT')</code>	求解潮流，返回越限与电压违规
<code>analyze_contingencies(option="N-1", validate=False)</code>	N-1/N-2 故障分析
<code>get_power_flow_results(object_type, additional_fields)</code>	取指定对象类型（bus/gen/load/shunt/branch）结果
<code>get_key_field_list(object_type)</code>	查某对象类型有哪些可用字段
<code>change_parameters_multiple_element(...)</code>	批量修改同类型元件参数
<code>change_and_confirm_params(...)</code>	修改参数并验证是否生效
<code>get_ybus(full=False)</code>	母线导纳矩阵（完整/稀疏）
<code>get_jacobian(full=False)</code>	潮流雅可比矩阵
<code>get_lodf_matrix(precision=3, method='DC')</code>	线路开断分布因子矩阵
<code>get_ptdf_matrix_fast()</code>	功率转移分布因子矩阵（快速算法）
<code>to_graph(node='bus', geographic=False)</code>	转成 NetworkX 图
<code>determine_shortest_path(start, end, ...)</code>	两母线间最短路径
<code>run_robustness_analysis()</code>	系统鲁棒性分析

实践提示： `change_and_confirm_params` 比 `change_parameters_multiple_element` 更值得优先使用——它会确认改动真的生效了，避免“以为改了但其实没改”的静默失败。这在自动化流程里非常关键。

4.2 PSS/E（含独门绝技：命令动态桥接）

PSS/E 有一个别的工具都没有的能力——不只是暴露固定工具，还让模型自己能查、能调 PSS/E 的底层命令：

层级	工具	说明
基础	<code>open_case(case)</code>	打开案例
基础	<code>solve_case()</code>	求解潮流
发现	<code>search_psspy_commands(query, category)</code>	按关键词搜索 PSSPY 命令
发现	<code>lookup_psspy_command(function_name)</code>	查某个命令的完整签名
执行	<code>run_psspy_command(function_name, arguments)</code>	执行任意 PSSPY 命令

这是 PowerMCP 里最重要的设计选择之一。别的工具只能做开发者预设的那几十件事；PSS/E 这条链路等于把 PSS/E 数千个 API 全部开放给模型。代价是需要模型先搜索、再查签名、最后执行——但能力上限完全不同。

使用纪律（来自官方技能文档）：

- 不要把 run_psspy_command 当搜索工具用——先 search 或 lookup
- 每条执行的命令都要绑定一个研究目标，而不是"试试这个 API"

安全设计（值得知道）：PSS/E server 内置 22 条命令禁止清单，拦截"启动任意程序、执行命令文件、注入 Python 代码、加载原生库、读取含 Python 的 RAS 文件、I/O 重定向"等类别。拦截点早于引擎启动。设计动机写在源码注释里：路径围笼并不能让可执行输入变安全。此外 PSS/E 的 2184 份 PSSPY 命令规格以 JSON 形式随包分发，是本仓最大的文件量来源。

4.3 surge（开源输电网全流程，44 个工具）★

如果不想买商业许可，surge 是目前覆盖最全的开源选择：

案例 I/O → 潮流(AC/DC/FDPF) → 灵敏度(PTDF/LODF/OTDF)
→ OPF(DC/AC/SCOPF) → N-1/N-2 故障分析
→ NERC/AC 可用输电能力 → SCED/SCUC 调度
→ 检查/导出/图分析 → 网络构建与编辑

系统依赖（容易踩坑，先装好）：

平台	命令
Ubuntu / Debian	<code>sudo apt install libhighs-dev coinor-libipopt-dev</code>
macOS	<code>brew install highs ipopt</code>
Windows（实测可行路径）	<code>conda create -n powersolvers -c conda-forge highs ipopt</code> ，再把库目录指给 <code>HIGHS_LIB_DIR</code>

关键提醒：pip install highspy 单独安装不满足 HiGHS 的运行时链接要求，必须装系统库。库在非标准路径时，设置 `HIGHS_LIB_DIR` 和 `IPOPT_LIB_DIR`。Gurobi、COPT、CPLEX 是运行时插件——有许可就自动检测使用，没有则回退到 HiGHS/Ipopt。

Windows 上的实测边界（重要）：

- **HiGHS 可靠可用**：设好 `HIGHS_LIB_DIR` 后 DC OPF 连续 3 次运行结果完全一致（cost=125948.01），确定性通过。
- **Ipopt 未能可靠解决**：试遍别名 / `IPOPT_LIB_DIR` / `PATH` / 预加载共 7 种组合均失败；有一次不可复现的成功，另一次尝试中进程静默崩溃。故 **AC OPF 在 Windows 上暂不可用**，需要 AC OPF 时请走 pandapower 的 `runopp`（见 6.3 节）。
- Gurobi 路径另有**两重阻塞**：surge 只支持 Gurobi 13.x，而本机为 11.0.3；且许可已于 2026-03-31 过期。改 DLL 名行不通——旧版本 API 不兼容，会在符号解析阶段崩溃。

4.4 ANDES（动态分析）

工具	作用
<code>run_power_flow(file_path, dyr_path=None)</code>	潮流求解，可选附加 .dyr 动态模型文件
<code>run_time_domain_simulation(step_size=0.01, t_end=10.0)</code>	时域仿真
<code>run_eigenvalue_analysis(file_path)</code>	特征值（小信号）分析
<code>get_system_info()</code>	系统信息

`run_power_flow` 返回的两个关键字段（很容易被忽略，但很重要）：

- `n_dynamic_generators`：实际加载的动态发电机模型数量（汇总 ANDES 的 SynGen、RenGen、DG 三组）
- `dynamic_models_loaded`：布尔值

这是一个设计得很诚实的细节：`dynamic_models_loaded` 是根据实际加载了什么推导出来的，而不是根据你是否传了 `dyr_path`。原因是 ANDES 的 PSS/E dyr 解析器会跳过它不支持的模型类型——所以即使你传了 `.dyr`，系统仍可能没有任何动态模型。反过来说，像 `kundur_full.json` 这种自身内嵌动态的案例，不传 `.dyr` 也会报告 True。

实践含义：想跑时域仿真或特征值分析前，必须先确认 `dynamic_models_loaded` 为 True，否则你算的是一个没有动态的静态拓扑。

4.5 PyPSA（规划与优化）★

工具	作用
<code>load_network(file_path)</code> / <code>create_network(...)</code>	加载或新建网络
<code>get_network_info(network_name)</code>	网络统计
<code>get_component_details(...)</code>	元件详情
<code>run_power_flow(network_name, linear=False)</code>	潮流（可行性检查）

工具	作用
<code>optimize_network(...)</code>	运行/调度优化
<code>optimize_investment(...)</code>	容量扩张优化
<code>add_bus</code> / <code>add_generator</code> / <code>add_load</code> / <code>add_line</code> / <code>add_storage_unit</code>	网络构建
<code>import_from_csv_folder</code> / <code>export_to_csv_folder</code>	CSV 导入导出
<code>import_case_from_json(powerio_ir=..., output_path=...)</code>	经 PowerIO IR 导入

环境要求：PyPSA $\geq 0.35.2$ 且 < 2 ；至少一个 LP 求解器（推荐开源 HiGHS；商业可用 Gurobi 学术版）。

两个实测陷阱：

- **不要传 `linearized=True`**：PyPSA 1.2.4 的 `optimize()` 没有这个参数，实参落进 `**kwargs` 被静默忽略，你以为在跑 DC OPF，实际跑的是 AC OPF。关键参数请先核对函数签名。
- **不要读 `Network.status`**：该属性在 PyPSA 1.2.4 中已移除。用 `hasattr` 判断会得到 `False`，从而误报“未收敛”——而求解器实际返回的是 `('ok', 'optimal')`。正确做法是从 `optimize()` 的返回值取状态。

4.6 pandapower（稳态 AC）★

工具	作用
<code>load_network(file_path)</code> / <code>create_empty_network()</code>	加载/新建
<code>get_network_info()</code>	统计与数据
<code>run_power_flow(algorithm, ...)</code>	潮流（Newton-Raphson / 前推回代）
<code>run_contingency_analysis(contingency_type, elements)</code>	N-1/N-2 故障分析
<code>load_network_from_any</code> / <code>load_network_from_json</code>	通过 PowerIO 导入
<code>export_network_to_format(to_format)</code>	通过 PowerIO 导出

版本兼容提醒：`run_contingency_analysis` 在 pandapower 3.5.4 下会因 `net.deepcopy()` 已被移除而必然失败。该缺陷源于上游 `requirements.txt` 未钉版本，且上游 `tests/` 中没有该工具的功能测试，因此 CI 未拦截。修复方式是把两处 `net.deepcopy()` 改为 `copy.deepcopy(net)`（1 文件 +3 -2）；未修复时应改用 `surge` 做 N-1。另需注意：该工具即便可用也只返回**元件索引列表**，不含 `vm_pu` / `loading_percent` 数值，且分析后不保留故障态。

另有一个能力缺口值得知道：pandapower server 的 8 个工具中**没有 OPF**，与 PyPSA 的能力不对称。需要 AC/DC OPF 时请走 6.3 节给出的路径。

4.7 OpenDSS（配电网）

本节的工具名已更正（第一版所载 6 个名称均不存在）：经与 OpenDSS server 的 55 个工具逐一比对，第一版列出的 `compile_and_solve`、`get_bus_voltages`、`get_total_power`、`set_load_multiplier`、`run_daily_energy_meter`、`get_harmonic_results` 全部不存在。照旧名调用会直接失败。下表为源码提取的真实工具名。

真实工具名	作用
<code>compile_opendss_file(...)</code>	编译 DSS 文件
<code>solve_snapshot()</code>	求解快照（与上者配合完成"编译并求解"）
<code>get_voltage_mag_ln_nodes_records(...)</code>	节点电压幅值（另有 <code>ll / smart</code> 变体）
<code>get_powers_p_records(...)</code> / <code>get_powers_q_records(...)</code>	有功 / 无功记录

OpenDSS 共 55 个工具，构成为：`model.py` 27 + `results.py` 23 + `configuration.py` 2 + `interactive_view.py` 2 + `simulation.py` 1。其中没有任何谐波类工具；负荷倍率与日能量计量也未找到对应项。此外 OpenDSS 使用工厂模式注册工具（`create_mcp()`），不是装饰器风格——任何基于 `@mcp.tool` 的静态统计都会漏掉它。

4.8 Egret（市场与优化）

三层递进，从最便宜能回答问题的模型开始：

`solve_dc_opf()`
 ↓ 需要电压/无功真实性时
`solve_ac_opf()`
 ↓ 需要时段、备用、启停逻辑时
`solve_unit_commitment_problem()`

← 先做，快速可行性与阻塞筛查

4.9 其他工具

工具	关键能力
PowerFactory	<code>connect</code> 、 <code>activate_study_case</code> 、 <code>run_loadflow</code> 、 <code>run_rms_simulation</code> 、 <code>export_results_to_csv</code> 、 <code>export_project_to_pfd</code> （共 22 个工具）
PSCAD	基于 AST 解析 mhi-pscad 库的装饰器与类型提示，给 AI 深度 API 感知；含 30 秒执行看门狗防止挂死（共 26 个工具）
LTSpice	网表生成、无头仿真、波形选择与绘制（共 7 个工具）
GenX	<code>plot_capacity</code> （按类型聚合绘图）、 <code>check_capacity_setting</code> （brownfield/greenfield 识别）、 <code>summarize_capacity</code> （共 7 个工具）

工具	关键能力
HOPE	双入口：本地 stdio 全权限 + 远程只读 HTTP（供 ChatGPT 网页开发者模式）；只读模式拒绝非回环地址绑定（共 20 个工具）
PLEXOSDB	29 个对象/属性/成员 CRUD + <code>translate_to_sienna</code> + <code>compare_solutions</code> ；无需 PLEXOS 许可或安装（本仓 2 个自有工具 + 29 个上游工具）

五、核心机制：PowerIO 跨软件案例翻译

这是 PowerMCP 工程上最有价值的设计，值得单独一章。

5.1 问题：每种软件说自己的"语言"

`.raw`（PSS/E）、`.pwb` / `.aux`（PowerWorld）、`.dss`（OpenDSS）、`.nc`（PyPSA）、`.m`（MATPOWER）……格式各不相同。过去要跨软件分析，只能靠人工导出导入。

5.2 解法：一个统一的 IR 传输格式

PowerIO 作为核心依赖（不是可选组件）内置于 PowerMCP，承担"翻译层"职责。交换格式是 **PowerIO IR generation 2**（`"schema": "pio-ir"`，`"version": 2`）——一个带类型、带来源信息、带诊断的文档：

```
parsed = parse(path="case9.raw")           # powerio server: 解析成 IR
ir = parsed["powerio_ir"]

summarize(powerio_ir=ir)                   # 摘要：元件数 / 拓扑 / 参考母线
diagnostics(powerio_ir=ir)                 # 诊断：code / severity / target / 建议动作
calc_matrix(matrix="ptdf", powerio_ir=ir) # 直接建矩阵（行列带母线/支路标识）

load_network_from_json(powerio_ir=ir)      # → pandapower
import_case_from_json(powerio_ir=ir, output_path="case9.nc") # → PyPSA
load_model_from_json(powerio_ir=ir)        # → Egret
load_network_from_json(powerio_ir=ir, out_path="case9.m")   # → ANDES
emit(format="psse", destination="case9.raw", powerio_ir=ir) # → 落盘给只认路径的软件
```

核心思路：解析一次，在工具调用之间传递返回的 IR 字符串，只在后端确实需要文件时才落盘。`network_json` 是 `powerio_ir` 的别名，两者等价。

边界（官方明确声明）：PowerMCP 自身**从不重新解析、重新校验或重新计算** PowerIO 所声明的任何东西——它只负责把已声明的值路由给能接受它的消费者。

5.3 能构建的矩阵

PowerIO 可直接构建求解器需要的稀疏矩阵：

矩阵	用途
B'、B''	快速解耦潮流
Y_bus	节点导纳矩阵
PTDF	功率转移分布因子
LODF	线路开断分布因子
Laplacian	图拉普拉斯矩阵
LACPF	线性 AC 潮流

5.4 类型化编辑 (edits)：求解器导入前的结构化变更

adapter 接受 `edits` 参数——一个 JSON 列表，描述要在转换前应用的 what-if 变更。**整批先校验、再按列表顺序应用**；同一类的连续更新作为一个原子批次；母线负荷重分配会看到编辑产生的值。

op	键
<code>set_load_active_power</code>	<code>load</code> 、 <code>mw</code> 、 <code>terminal?</code>
<code>set_load_reactive_power</code>	<code>load</code> 、 <code>mvar</code> 、 <code>terminal?</code>
<code>set_generator_active_power</code>	<code>generator</code> 、 <code>mw</code> 、 <code>terminal?</code>
<code>set_generator_reactive_power</code>	<code>generator</code> 、 <code>mvar</code> 、 <code>terminal?</code>
<code>set_generator_voltage_magnitude</code>	<code>generator</code> 、 <code>vm_pu</code>
<code>set_generator_in_service</code>	<code>generator</code> 、 <code>in_service</code>
<code>set_branch_in_service</code>	<code>branch</code> 、 <code>in_service</code>
<code>set_transformer_tap_ratio</code>	<code>transformer</code> 、 <code>tap_ratio</code>
<code>set_transformer_phase_shift</code>	<code>transformer</code> 、 <code>shift_degrees</code>
<code>set_switch_closed</code>	<code>switch</code> 、 <code>closed</code>
<code>set_branch_thermal_rating</code>	<code>branch</code> 、 <code>mva</code> 、 <code>terminal?</code>
<code>set_bus_load_active_power</code>	<code>bus</code> 、 <code>mw</code> 、 <code>allocation</code> (<code>proportional_to_current_active_power</code> 或 <code>equal</code>)

元件 id 使用 PowerIO 报告的稳定标识（`loads:0`、`branches:3` 或源 uid）。母线负荷编辑必须指定分配规则——因为 PowerIO 从不把聚合需求分配给任意一个负荷。

```
load_network_from_json(
    powerio_ir=ir,
    edits='[{"op": "set_load_active_power", "load": "loads:0", "mw": 91.5},'
    ' {"op": "set_branch_in_service", "branch": "branches:3", "in_service": false}]',
)
```

5.5 一个体现工程严谨性的设计：拒绝静默求解

这是 PowerMCP 里最值得称赞的一个安全设计。

场景：包内存储了多个运行状态，但调用方没有指定用哪个。

```
# 静态包可以直接加载
import_case_from_json(powerio_ir=pkg, output_path="case9.nc")

# 含多个存储状态的包，必须显式选择
import_case_from_json(
    powerio_ir=pkg,
    output_path="dispatch.nc",
    scenario_id="high-demand",    # ← 必须显式指定
    time_index=3,                # ← operating_point 是它的别名
)

load_network_from_json(powerio_ir=pkg, study_commit=1)    # pandapower
```

如果调用方不选，PowerMCP 会报错，而不是默认求解基础模型。同样的保护还有两处：

保护机制	行为	防止的问题
未选状态拒绝	含多状态包未指定 → 报错	悄悄求解错的运行点
多导体包拒绝	平衡求解器拒绝多相包，除非显式降维	配电网到输电网的有损归约被隐式执行
路径预检	显式与旧式派生的 CSV 写入目标都做预检	越界写文件

为什么这很重要：在电力工程里，一个悄悄算错的结果比一个明确的报错危险得多。报错你可以修；算错了你可能直接拿去做决策。PowerMCP 选择了“宁可报错也不猜”——这是一个成熟的工程设计取向。

配电网场景下，多导体值会被平衡求解器拒绝，直到调用方主动要求转换：传 `to_balanced=True`（并给 `base_mva`），响应里会带 PowerIO 的就绪报告；也可以先调 powerio server 的 `to_balanced_report` 与 `to_balanced`。**保留某元件不等于所选求解器能建模它。**

5.6 路径沙箱

PowerIO 的 MCP 工具只接受本地路径和 `file://` URI，非本地 URI 方案一律拒绝。可通过环境变量限定可访问目录：

```
export POWERIO_MCP_ALLOWED_ROOTS="/data/cases:/home/user/projects"
```

策略实现不在本仓，而是直接复用 `powerio.mcp.sandbox` ——避免两份策略实现走样。关键性质是**先解析、后比较**，所以 `..` 段和指向根外的符号链接都逃不掉（“被比较的是真实目标，而不是拼写”）。

不设置这个变量的后果（实测）：路径围笼会退化为“服务进程启动目录”，`doctor` 对此报**黄色警告**而非错误。其注释直言：这个默认值“比操作者通常想要的更窄，而且没有写在任何地方”。所以只要你的算例不在 `server` 启动目录下，**读文件就会被拒**——这是策略问题，不是 bug。

诚实的技术说明（官方文档中明确写出）：这些是预检（preflight）检查，不是完全隔离。另一个进程理论上可以在检查之后、后端打开文件之前替换掉那个条目。这是“降低风险”而非“消除风险”。

除了路径围笼，本仓还有一层更硬的控制：**AST 静态强制检查**。测试用 `ast.parse` 解析每个 `server` 的源码，断言“每个接收模型路径参数的 tool 真的调用了 `checked_path`”。这不是单元测试，而是**契约一致性门禁**——新增一个接收路径的 tool 却忘了走围笼，CI 会直接红。

5.7 特殊案例：PowerWorld 显示文件

PowerWorld 的 `.pwd` 显示文件需要单独解码：

```
display(path="...")      # 返回图纸画布 + 每个变电站的显示坐标
```

注意：显示几何信息与 `.pwb` / `.aux` 的案例数据是两回事，不要混淆。

六、当前可用性全景（实测）

前面几章讲的是“工具能做什么”。本章回答一个更实际的问题：**在本机、现在，哪些真的能用？**

结论来自一次完整的实测：在隔离虚拟环境（Python 3.12.6）中安装 `powermcp 0.4.0`，然后逐个拉起 `server`、经 `stdio` 握手、调用 `list_tools` 取真实返回值，并在 IEEE 39 算例上实际跑通潮流、N-1、灵敏度与 OPF。

6.1 可用性分级定义

本机实测把 16 个 `server` 分为五级。**后续所有场景判断都以这五级为准：**

级别	含义	判据
可用（已验证）	装好即可调用，且有实际运行结果	运行时 <code>list_tools</code> 与静态统计一致 + 至少一次真实调用成功
可启动（未验证功能）	包已装、 <code>server</code> 能起，但未做功能验证	依赖 <code>ok</code>

级别	含义	判据
装得上 ≠ 跑得动	Python 包与工具已注册，但缺外部运行时	调用时才报错
未安装（按需可加）	有 pip extra，本次未装	doctor 报 missing
需商业软件/授权	依赖本机闭源软件与配置路径	doctor 报 vendor engine 未配置

6.2 服务器级总表

工具数为静态统计值；标 ★ 者已经过运行时 `list_tools` 核验。

Server	工具数	本机状态	级别	前置条件
surge ★	44	潮流 / PTDF / LODF / N-1 / N-2 / ATC 可用；DC OPF 需 <code>HIGHS_LIB_DIR</code>	可用	系统 C 库 HiGHS（非 pip 可装）
PyPSA ★	17	潮流、优化（原生算例）、导入导出可用	可用	需 LP/NLP 求解器在 PATH
powerio ★	10	parse / emit / summarize / diagnostics / calc_matrix / to_balanced 可用	可用	无
pandapower ★	8	潮流、建模、转换可用；N-1 需先修复 <code>deepcopy</code>	可用	无
HOPE ★	20	包与工具已就绪，缺 Julia 运行时	装得上 ≠ 跑得动	Julia + HOPE 仓库本体
GenX	7	包已就绪，缺 Julia	装得上 ≠ 跑得动	Julia + GenX.jl checkout
OpenDSS	55	未装 <code>[opendss]</code> extra	未安装	<code>pip install powermcp[opendss]</code>
ANDES	6	未装 <code>[andes]</code> extra	未安装	<code>pip install powermcp[andes]</code>
Egret	5	未装 <code>[egret]</code> extra	未安装	需 <code>pyomo</code> ，较重
LTSpice	7	未装 <code>[ltspice]</code> extra	未安装	<code>pip install powermcp[ltspice]</code>
PLEXOSDB	31	未装（该包未上 PyPI）	未安装	需手动 git 安装
PowerWorld	14	软件已装，但为 Education 版，无 SimAuto COM	需商业授权	商业授权版 + <code>esa</code> 包
PSSE	5	未配置	需商业授权	<code>psse.python_lib</code> / <code>psse.bin</code>
PSLF	12	未配置	需商业授权	<code>pslf.python_lib</code>
PowerFactory	22	未配置	需商业授权	<code>powerfactory.python_path</code>

Server	工具数	本机状态	级别	前置条件
PSCAD	26	未装 mhi-pscad	需商业授权	PSCAD 软件 + [pscad-windows]

关键比例：全仓约 **250 个工具**中，本机**实际可用 79 个**（4 个开源 server），约占三成。其余受"extra 未装"或"商业授权"限制。

运行时核验结果：pandapower 8 / pypsa 17 / surge 44 / hope 20 / powerio 10，与静态统计**逐项吻合**。这一步很必要——静态统计曾经漏掉 OpenDSS 与 PSCAD 两个 server（它们不用装饰器注册），核验把其中 5 个 server 的数字从"推断"升级为"实测"。

6.3 开箱可用的四大能力支柱

支柱	状态	路径	实测证据
① 基态潮流	可用	MCP	pandapower / PyPSA / surge 三引擎结果一致
② 跨格式解析与矩阵	可用	MCP (PowerIO IR)	跨求解器保真达机器精度
③ PTDF / LODF 与 N-1	可用	MCP (surge)	46 场景扫描，带完整数值
④ AC / DC OPF 与容量规划	可用	Skill 侧 Python (runopp / rundcopp)，非 MCP	三项守恒校验残差 0.0000 MW

第 ④ 项必须单独说明：AC/DC OPF **不是通过 MCP 工具调用的**。原因是两条 MCP 路径都不可用——pandapower server 没有 OPF 工具，而 PyPSA 的 optimize_network 对经 PowerIO IR 导入的算例返回 infeasible（已排除 13 项假设，**根因未定位**）。因此改用 pandapower 自带的 runopp（AC）/ rundcopp（DC），它们与已验证的 runpp 同引擎、同网络对象、同索引约定，不引入新的数据契约。

为弥补这一弱点，示例脚本设置了一**致性闸门**：从与 MCP 相同的算例文件本地重建网络，逐母线比对基态潮流电压，最大偏差超过 1e-06 pu 就中止——保证 OPF 的结论对 MCP 侧成立。实测闸门偏差为 **0.000e+00 pu**。

6.4 实测数值（IEEE 39 算例）

算例规模：**39 母线 / 35 线路 / 11 变压器 / 9 机组 + 1 平衡机 / 21 负荷**

指标	基态潮流	AC OPF	DC OPF
发电合计 (MW)	6297.8711	6298.4106	6254.2300
网损 (MW)	43.6411	44.1806	0.0000（模型无网损）
最低电压 (pu)	0.9820（母线 30）	0.9820	—
最高电压 (pu)	1.0636（母线 35）	1.0600（触上限）	—

指标	基态潮流	AC OPF	DC OPF
线路最大负载率 (%)	73.3709 (线路 21)	87.6447	90.16
目标函数值	(未优化)	41,872.30	41,263.94

AC 与 DC 目标值相差 **608.36 (+1.474%)** ——AC 计及 44.18 MW 网损、DC 不计，故 DC 目标值更低属预期。三项守恒律校验（基态、AC OPF、DC OPF）残差均为 **0.0000 MW**。

N-1 与灵敏度 (surge)：

- N-1 扫描：**46 场景 / 23 有越限 / 52 条越限**；线路负载率 >100% 的故障 19 个
- Top-1 `branch_26` (Line 21→22) **161.84%**，`flow = 958.49 MW`；Top-2 `branch_17` 133.50%；Top-3 `branch_37` 127.59%
- 母线电压 <0.95 pu 的故障 2 个，最低 `min_vm_pu = 0.9361`
- PTDF 46 × 39，非零元 1090（稀疏度 0.3924）；LODF 46 × 46，非零元 763，Inf/None 506
- 物理自治性核对通过**：LODF 对角线 = -1.0；同母线并联支路互为相反数 ±1.0

跨格式保真 (PowerIO IR)：同一份 IR 不经文件分别喂给 pandapower 与 PyPSA，母线电压最大偏差 $|\Delta| = 6.98\text{e-}11$ pu，支路有功最大偏差 $|\Delta| = 2.25\text{e-}07$ MW——往返语义保真度**达到机器精度**。

一个必须避开的对比陷阱：

两侧元件编号约定不同——pandapower 的 `res_bus / res_line` 用 **0-based 整数索引**，pypsa 沿用 IR 里的 **1-based 标识**（母线 `"1" .. "39"`，线路 `"line_1" .. "line_35"`）。按字面键 join 等于拿不同物理元件对比（首次对比曾误报 0.068 pu 的假差异）。正确做法是**自动探测编号偏移**（搜索使最大残差最小的偏移量），而非硬编码。

6.5 商业 / 高依赖 Server 的探查结论

这一节的结论对排期最有价值：**软件存在 ≠ 可自动化**。

Server	探查结果	根因
PowerWorld	软件已安装（ <code>Simulator GOS Education 23</code> ），但 SimAuto COM 未注册、 <code>esa</code> 包未装	<code>esa</code> 通过 COM 驱动 SimAuto， Education/GOS 版不注册 SimAuto COM 类 ——即便装上 <code>esa</code> 也无法工作
HOPE	PyYAML 已装，工具可注册；缺 Julia 运行时与仓库本体	求解需调用 Julia 侧引擎
GenX	matplotlib / pandas 已装；缺 Julia 与 GenX.jl checkout	同上
PSSE / PSLF / PowerFactory	未配置厂商引擎路径	需各自商业授权与安装
PSCAD / LTSpice / PLEXOSDB	缺对应 pip 包	PLEXOSDB 的包未上 PyPI，需手动 git 安装

这是 PowerMCP 的普遍模式：装得上 ≠ 跑得动。 闭源/外部引擎的 server 在包层面可导入、工具也能注册，但真正调用工具时才报错。全仓 15 个 server 的生产代码中**没有任何 mock / stub 降级路径**（仅测试里用 `unittest.mock`），统一做法是"延迟导入 + 可操作报错"。

对使用者的含义：不要假设工具可用。**开工前必须先做可用性预检**——先跑 `powermcp doctor`，再对目标 server 做一次真实的依赖探测（软件是否安装 → COM/授权是否注册 → 桥接包是否就位）。

6.6 已知缺陷与当前处置

问题	影响	当前处置
pandapower 的 <code>run_contingency_analysis</code> 调用已移除的 <code>net.deepcopy()</code>	该工具在 pandapower 3.5.4 下必然失败	已修复（1 文件 +3 -2），实测 46 故障 / 45 越限；未向上游提 PR
PyPSA 的 <code>optimize_network</code> 对 IR 导入算例返回 infeasible	PyPSA 的 OPF 路径在 IR 导入场景不可用	已排除 13 项假设， 根因未定位 ；改用 pandapower <code>runopp</code> 绕过
PyPSA 无 <code>linearized</code> 参数，实参被静默忽略	早期"DC OPF 亦不可行"的结论作废	已更正；DC 路径需另寻
两侧元件编号约定不同	字面键对比会拿错物理元件	改为自动探测偏移
surge 的 AC OPF 需 Ipopt 或 Gurobi 13	Windows 上 AC OPF 不可用	AC OPF 走 pandapower <code>runopp</code> （不受影响）
工具名跨 server 严重冲突	多 server 共存时模型无法靠名字消歧	编排时必须 显式指定目标 server

关于工具名冲突，值得展开说： `load_network` 出现在 pandapower / PyPSA / surge；`run_power_flow` 出现在 pandapower / PyPSA / PowerWorld；`run_contingency_analysis` 出现在 pandapower / PyPSA / PowerWorld / PowerFactory；`add_bus` / `add_generator` / `add_load` / `add_line` 更是分布在 5 个 server。

这不是缺陷，而是多 server 聚合的固有代价。但含义很直接：若把多个 server 挂进同一个客户端命名空间，模型面对 `load_network` 时无法从名字判断该调用哪一个——不同 server 的同名工具语义、参数、单位可能都不同。

七、实战示例

示例 1：完整的 N-1 安全分析（开源工具链）

目标：分析 IEEE 39 节点系统的 N-1 安全水平。

准备：

```
pip install powermcp
powermcp install      # 勾选 pandapower + PyPSA + PowerIO
powermcp doctor       # 确认环境正常
```

对话 (Claude Desktop / Claude Code / Cursor):

```
你: 加载 case39.json, 先跑基态潮流, 然后做 N-1 分析,
    把过载元件和电压越限母线都列出来。

模型: 调用 load_network("case39.json")
      → 调用 get_network_info()
      返回: 39 母线、35 线路、11 变压器、9 机组 + 1 平衡机、21 负荷
      → 调用 run_power_flow()
      返回: 收敛
      → 调用 run_contingency_analysis()
      返回: N-1 排序结果
```

实测输出:

```
【基态潮流】收敛
最低电压: 母线 30 = 0.9820 pu (下限 0.95 pu)
最高电压: 母线 35 = 1.0636 pu
最重载线路: 线路 21 = 73.37% (限值 100%)
过载线路: 0 条
网损 43.6411 MW | 总负荷 6254.23 MW | 总发电 6297.87 MW
[守恒校验] 发电 - (负荷 + 网损) = 0.0000 MW

【N-1 分析】46 场景 / 23 有越限 / 52 条越限
线路负载率 > 100% 的故障: 19 个, Top-3 为
  branch_26 Line 21→22 161.84% flow 958.49 MW
  branch_17 Line 13→14 133.50%
  branch_37 Line 10→32 127.59%
母线电压 < 0.95 pu 的故障: 2 个, 最低 0.9361 pu (branch_42)
```

关键观察: Top-3 故障都指向同一片区域 (母线 21~24 一带), 这提示可以用一个统一措施覆盖整族故障, 而不是逐个处理。这正是 PowerSkills 技能里"按共同走廊/断面分组"的思路。

一个解释纪律 (避免误读): 母线 35 的 1.0636 pu **恰等于该处发电机的电压设定值**——它是 PV 母线, 由机组维持在设定点。所以"有 7 个母线高于 1.05 pu"反映的是**算例数据的设定值偏高**, 而不是求解异常。报告措辞必须区分这两者。

示例 2: 用 PSS/E 的命令桥接做自定义分析

场景: 你需要执行一个 PSS/E 标准工具没覆盖的操作。

你：我想在 PSS/E 里改一下 G3 的机端电压目标值，
先查一下应该用哪个 PSSPY 命令。

模型：调用 `search_psspy_commands(query="voltage setpoint")`
→ 返回候选命令列表，例如 `change` 相关命令
调用 `lookup_psspy_command(function_name="...")`
→ 返回完整签名、参数个数与类型、错误码说明
调用 `run_psspy_command(function_name="...", arguments=[...])`
→ 执行
调用 `solve_case()`
→ 重新求解，确认效果

注意这个流程的三段式：搜索 → 查签名 → 执行。不要跳过前两步直接执行——PSSPY 命令的参数顺序和数据类型很严格，猜错会得到无意义的错误码。

示例 3：跨软件案例翻译

场景：你有一个 PSS/E 的 `.raw` 案例，但想用 pandapower 快速筛查。

你：解析 `case9.raw`，然后导入 pandapower 跑潮流。

模型：调用 `parse(path="case9.raw")`
→ 返回 `{"powerio_ir": "...", "summary": {...}}`
`summary.elements.buses = 9`
`summary.elements.branches = 9`
`summary.topology.connected_components = 1`
调用 `load_network_from_json(powerio_ir=<上面的 IR>)`
调用 `run_power_flow()`

同一个 IR 可以继续给别的工具用：

```
调用 import_case_from_json(powerio_ir=<同一个 IR>, output_path="case9.nc") # → PyPSA
调用 calc_matrix(matrix="ptdf", powerio_ir=<同一个 IR>) # → 建 PTDF
调用 emit(format="psse", destination="out.raw", powerio_ir=<同一个 IR>) # → 落盘
```

效率提示：解析一次，多处复用。不要为每个工具重新 `parse`。

实测验证：同一份 IR 分别喂给 pandapower 与 PyPSA，母线电压最大偏差 **6.98e-11 pu**、支路有功 **2.25e-07 MW**——转换不引入可观测误差。但跨引擎对比时务必先解决编号偏移问题（见 6.4 节）。

示例 4：动态稳定性筛查（ANDES）

场景：检查系统的振荡模式阻尼是否达标。

你：对 IEEE 14 节点案例做特征值分析，看看有没有阻尼不足的模式。

模型：调用 `run_power_flow("ieee14.xlsx")`
→ 基态求解完成，14 母线、5 发电机
调用 `get_system_info()`
→ 确认动态模型、励磁与调速器类型
调用 `run_eigenvalue_analysis("ieee14.xlsx")`
→ 返回 `modes` 列表（已按阻尼最差优先排序）

输出解读：

主导模式：频率 0.42 Hz，阻尼比 3.1%

工程判据：阻尼比低于约 5% 通常视为需要关注。3.1% 说明这个案例的阻尼不足，应该转给动态稳定性处置流程。

重要提示： `run_eigenvalue_analysis` 返回的 `modes` 列表是**重新排序过**的（按最差阻尼优先），但 `participation_factors` 矩阵**没有重排**，仍是 ANDES 原生顺序。查参与因子时要用 `mode` 的 `index` 字段，而不是它在 `modes` 列表里的位置。这是一个容易踩的坑。

示例 5：配电网 DER 接入筛查（OpenDSS）

你：编译 `feeder.dss`，跑一下潮流，看看电压分布。

模型：调用 `compile_opendss_file(...)`
调用 `solve_snapshot()`
调用 `get_voltage_mag_ln_nodes_records(...)`
→ 返回各母线电压，识别薄弱点

扩展场景（负荷增长筛查）：修改负荷后重新求解，对比电压变化。注意 `set_load_multiplier` 这类旧名工具**并不存在**（见 4.7 节的更正），请以 `list_tools` 返回的真实工具名为准。

示例 6：从案例文件保存为其他格式

场景：OpenDSS 需要 `.dss` 文件，但你手上是别的格式。

调用 `emit(format="dss", destination="feeder.dss", powerio_ir=..., ...)`
调用 `compile_opendss_file(dss_file="feeder.dss")`

说明：OpenDSS 输出是一个**目录包**（bundle），其主 DSS 产物正是 `compile_opendss_file` 接受的输入。`bmopf-json@0.1.0` 与 `bmopf-json@0.2.0` 用于选择 BMOPF schema 版本，`geo-json` 写出地理图层。

另需注意：`Package`、`model-json`、`package_json` 与包格式的 `study_commit` 均为**已退役格式**，需要迁移——重新解析原始算例并传入其 IR。

八、完整使用流程：从零分析一个新算例

第七章的示例都是“已知算例、已知工具”的短流程。本章给出一个更贴近实际工作的完整流程：**手上有一个全新算例（来源与格式未知），如何从头分析到出报告**，并给出每一步可直接复制使用的提示词。

8.1 场景设定与三个硬前提

场景：某个目录（本章以 `D:\GridData\` 为例）下放入了一份全新算例，格式未知——可能是 PSS/E `.raw`、MATPOWER `.m`、pandapower `.json`、PyPSA `.nc`、OpenDSS `.dss` 或 PowerWorld `.pwb`。要求从零走完全流程，最终交付一份可复核的分析报告。

#	硬前提	为什么	检查方式
1	运行环境就绪	没有 powermcp 则一切无从谈起	<code>powermcp --version / doctor</code>
2	路径围笼包含目标目录	PowerIO 路径策略默认约束到“服务进程启动目录”，读不到别处的文件； <code>doctor</code> 只报黄不拦截	设置 <code>POWERIO_MCP_ALLOWED_ROOTS</code>
3	求解器就位	surge 的 DC OPF 需系统 C 库 HiGHS（非 pip 可装）	设 <code>HIGHS_LIB_DIR</code> 后试跑一次

前置条件不满足时的处理约定：明确告警并给出修复步骤，**不要静默卡死、不要盲目重试**。推荐格式：

⚠️ 前置条件不满足：[具体问题]。修复：[具体步骤]

工具名免责声明：下文的工具名均取自实测。但已有先例——OpenDSS 技能引用的 **6 个工具名全部不存在**。因此**每一步执行前都应先以 `list_tools` 的实际返回为准**（见步骤 3）。

8.2 全流程地图

六个阶段、13 步，严格遵循 PowerSkills 的**渐进式披露**原则：载入 → 检查 → 基态 → 修改 → 高级研究。**不允许跳过基态直接做故障分析**。

阶段	步骤	目的	关键产出	风险
A 环境就绪	1-3	确认能跑、能读、工具名对	可用性清单	低（只读）
B 算例解析	4-5	拿到可信的统一表示 + 诊断	PowerIO IR + 诊断报告	低（只读）
C 基态与校验	6-7	建立基准 + 跨引擎交叉验证	潮流结果 × 2 引擎	低（只读）
D 灵敏度与故障	8-9	找瓶颈与关键故障	PTDF/LODF + N-1 排序	中
E 缓解与优化	10-12	给出措施 + 成本最优	措施对照表 + OPF	中
F 归档	13	留痕、可复现	分析报告	低

8.3 逐步详解（含提示词）

步骤 1 — 环境预检

目的：先确认哪些 server 真能跑，避免在不可用的路径上浪费时间。

先用 `powermcp doctor` 检查本机环境，然后给我一份表格，列三组：

- (1) 依赖 ok 的 server；
- (2) 报 missing 的 server 及缺失的包名；
- (3) 报 vendor engine / 需要配置路径的 server。

先只做检查，不要安装任何东西、不要改配置。
另外单独告诉我 surge 的 DC OPF 是否可用（看 `HIGHS_LIB_DIR` 是否已设）。

验收判据：能明确区分"可用 / 装得上 ≠ 跑得动 / 需商业授权"三类。

步骤 2 — 打通路径围笼（最容易漏的一步）

目的：让 server 能读到目标目录下的文件。

我后面要分析的算例放在 `D:\GridData` 下。
请先确认 `POWERIO_MCP_ALLOWED_ROOTS` 环境变量当前是否已设置、是否包含 `D:\GridData`。
如果没设置或不包含，只告诉我应该怎么设置，先不要动系统环境变量、不要改配置文件。
同时说明：不设置的话，哪些操作会失败、失败时的报错大概是什么样。

验收判据：明确知道围笼当前状态 + 是否需要设置。

纪律：修改系统环境变量属高危操作，**必须用户确认后**才执行。若报"路径不在允许根内"，说明围笼未包含目标目录——这是策略问题，不是 bug。

步骤 3 — 盘点目录 + 工具名核对（防坑双检）

目的：①搞清算例有哪些文件；②确认后面要用的工具名真实存在。

两件事，都只读不改：

第一，列出 `D:\GridData` 下的全部文件（含大小、扩展名、修改时间），并判断哪些是算例文件、哪些是附属数据（如负荷曲线、故障表、地理信息）。如果有多个候选算例，按"最可能是主算例"排序并说明理由，先不要解析。

第二，对 `powerio`、`pandapower`、`pypsa`、`surge` 四个 server 各调一次 `list_tools`，把工具名清单完整给我。我要用这份清单核对我后面要调的工具名是否真实存在。

验收判据：拿到文件清单 + 四份真实工具名清单。

步骤 4 — 解析为 PowerIO IR（统一数据契约）

目的：把任意格式的算例转成跨 server 通用的统一表示。后续所有引擎都从这一份 IR 出发，避免"各引擎各读各的格式"导致结论不可比。

用 powerio server 的 parse 工具解析 D:\GridData\<第 3 步确定的主算例文件>。

要求：

- 1) 把返回的 powerio_ir 完整保存下来（后面每一步都要复用同一份，不要重新解析）；
- 2) 告诉我返回的 schema 与 version；
- 3) 告诉我 value_type 是什么（BalancedNetwork 还是多导体/其它）；
- 4) 如果返回里带 diagnostics，先原样贴出来。

先不要做任何求解。

失败处理：若是**多导体网络**（配电网），平衡求解器会拒绝，需先调 `to_balanced_report` / `to_balanced` 降维；若是**已退役格式**，需重新 parse 原始算例。

步骤 5 — 摘要与诊断（先判断算例是否可用）

目的：在花时间跑潮流之前，先看这份算例本身有没有问题。这是最省钱的一步。

对刚才那份 IR 调 summarize 和 diagnostics，给我：

- A. 摘要：母线数 / 支路数 / 机组数 / 负荷数、连通分量个数、参考母线、是否辐射状。
- B. 诊断：逐条列出全部 diagnostics 记录（code、severity、target、suggested action）。

然后给结论：

- 如果 severity 里有 error 级别的问题，停下来，先告诉我要不要继续；
- 如果只有 warning，继续，但把 warning 记下来，后面解释结果时要带上。

为什么这一步重要：后面所有数值结论都建立在“算例可信”之上。算例数据自身不自洽是有先例的——IEEE 39 算例里机组 5 的电压设定值 1.0636 **高于**该母线上限 1.06。

步骤 6 — 基态潮流（建立基准）

目的：得到基准工况。后面所有的“越限”“恶化”都是相对这一基准而言的。

把这份 IR 用 pandapower 的 load_network_from_json 导入，然后 run_power_flow。
报告以下内容，全部要给具体数值和元件编号：

- 1) 是否收敛；
- 2) 最低电压：数值 + 母线编号；最高电压：数值 + 母线编号；
- 3) 最重载线路：编号 + 负载率（%）；
- 4) 低于 0.95 pu 的母线有几个、分别是哪些；
- 5) 高于 1.05 pu 的母线有几个、分别是哪些；
- 6) 过载（>100%）的支路有几个。

最后做一个守恒校验：总发电 - （总负荷 + 网损）= ？给出残差数值。

验收判据：收敛 = True；守恒残差 ≈ 0 （IEEE 39 实测为 **0.0000 MW**）；电压与负载率均有具体数值和元件编号——**不接受“有若干越限”这种模糊结论。**

步骤 7 — 跨引擎交叉校验（科研级必做）

目的：用第二个独立引擎复算，证明结果不依赖于某一个求解器的实现。

用同一份 IR（不要重新解析）调 pypsa 的 `import_case_from_json` 导入，再跑一次潮流。

然后做逐项对比：

- 母线电压 `vm_pu`：两边逐个母线比
- 支路首端有功：两边逐条支路比

关键：两侧的元件编号约定不同（pandapower 用 0-based 整数索引，pypsa 沿用 IR 里的 1-based 标识）。请自动探测编号偏移，搜索使最大残差最小的那个偏移量，不要按字面键直接对比——按字面比会拿不同的物理元件做对比，得出假差异。

给出：最大偏差、平均偏差、以及是否落在 $1e-06$ pu 的判定阈值内。

验收判据：编号偏移探测结果明确（IEEE 39 实测为“pypsa 键 = pandapower 键 + 1”）；最大偏差落在机器精度量级（实测 电压 $6.98e-11$ pu / 有功 $2.25e-07$ MW）。若出现 $0.0x$ pu 量级的“差异”，先怀疑编号错位，而不是物理差异。

步骤 8 — PTDF / LODF 灵敏度矩阵

目的：从“单点结果”升级到“结构性认识”——哪些线路对哪些注入最敏感。

用 surge 对这份算例计算 PTDF 和 LODF。

给我：

- 1) 两个矩阵的维度、非零元个数、稀疏度；
- 2) PTDF 里对几个关键母线注入最敏感的前几条支路；
- 3) LODF 里的 Inf / None 有多少个，并解释它们对应什么物理场景。

然后做物理自治性核对（这是正确性的证据，不是可选步骤）：

- LODF 对角线是否 = -1.0? 为什么？
- 同一对母线之间的并联支路，LODF 是否互为相反数？

验收判据：LODF 对角线 = -1.0（开断自身即全失）；同母线并联支路互为相反数；Inf/None 有物理解释（实测 506 个，对应开断后成孤岛、LODF 无定义）。

为什么自治性核对重要：矩阵算错了也能给出“漂亮的数字”。只有物理约束能证伪它。

步骤 9 — N-1 故障扫描（找关键故障）

用 surge 的 `run_n1_branch_contingency` 做 N-1 扫描。

给我：

- 1) 总共扫描了多少个场景、多少个场景出现越限、越限条目总数；
- 2) 线路负载率 $>100\%$ 的故障有几个，按最大负载率降序排前 5，
每个都要给出：断号、支路两端母线、最大负载率、对应 flow (MW)；
- 3) 母线电压 <0.95 pu 的故障有几个，最低电压是多少、在哪条支路；
- 4) 越限类型有哪些 (ThermalOverload / VoltageHigh / Islanding 等)。

注意：Islanding 类型可能没有支路定位信息，遇到时请容错标注，不要报错退出。

验收判据：Top-N 故障有断号 + 数值，不接受"有几个故障越限"这种不可排序的结论。

实测参照 (IEEE 39)：46 场景 / 23 越限 / 52 条越限；Top-1 `branch_26` (Line 21→22) **161.84%**、`flow = 958.49 MW`；低电压故障 2 个，最低 **0.9361 pu**。

步骤 10 — 越限 → 自动匹配缓解手册

目的：把"发现问题"推进到"知道下一步做什么"。这是 PowerSkills 的核心价值。

根据第 6 步和第 9 步的越限结果，对照 powerskills-tool 技能里的 Escalation triggers 表，告诉我每个越限应该匹配哪个 powerskills-engineering 缓解手册，并说明匹配依据是哪条具体规则。

预期映射：

- 母线电压 < 0.95 或 > 1.05 pu → voltage-violation-mitigation
- 支路负载率 $> 100\%$ → thermal-overload-mitigation
- N-1 有 binding contingency → contingency-mitigation
- 潮流不收敛 → convergence-failure-mitigation

然后按手册给出的顺序列出建议措施（先试什么、再试什么），
并说明每一项措施需要改动哪些元件、预期改善哪个指标。
先只给方案，不要动手改算例。

验收判据：每个越限都映射到具体手册 + 具体规则；措施有先后顺序，而不是一堆并列选项。

步骤 11 — 实施缓解措施并复算（用结构化变更）

目的：验证措施是否真的有效，且变更过程可审计。

我要试验第 10 步里的第 N 项措施。

请用 PowerIO 的 typed edits 把这次改动写成结构化的变更列表

(例如 set_branch_in_service / set_switch_closed / set_transformer_tap_ratio / set_generator_active_power 等)，而不是直接改文件。

要求：

- 1) 先告诉我你打算下发的 edits 列表 (op + 目标元件 + 新值)，等我确认；
- 2) 确认后再执行导入，并报告返回的 edits 报告 (按应用顺序列出被改动的元件)；
- 3) 在同一份新网络上重跑基态潮流 + N-1 扫描；
- 4) 给出"措施前 vs 措施后"对照表：最重载支路、最低电压、越限故障数。

注意：edits 是整批先校验、再按顺序应用的，请确认列表本身没有内部矛盾。

验收判据：返回的 edits 报告按应用顺序列出变更元件 (可审计)；措施前后对照表有具体数值。**若措施无效，如实报告无效**，不要换个说法让它看起来有效。

步骤 12 — 运行优化 (AC / DC OPF, 含一致性闸门)

现在做最优潮流，AC 和 DC 各做一次。

路径说明：本机 pandapower server 没有 OPF 工具，PyPSA 的 optimize_network 对 IR 导入的算例返回 infeasible (成因未定位)。所以这里走 pandapower 的 runopp / rundcopp，这是 Python 侧调用、不是 MCP 工具调用。

为保证结论对 MCP 侧成立，请先做一致性闸门：

- 1) 从与 MCP 相同的那个算例文件本地重建网络；
- 2) 逐母线比对基态潮流电压与第 6 步的 MCP 结果；
- 3) 若最大偏差 $> 1e-06$ pu，中止，不要在来历不明的网络上做优化。

闸门通过后给我：

- AC OPF：目标函数值、网损、电压范围、线路最大负载率；
- DC OPF：目标函数值、线路最大负载率；
- 两者目标值之差及其占 AC 的比例，并解释这个差是否合理；
- 达到出力上限的机组清单 (这是容量规划的直接依据)；
- 节点边际电价 lam_p 最高的前 5 个母线。

最后做守恒校验：AC 的"发电 - (负荷 + 网损)"、DC 的"发电 - 负荷"，给出残差。

验收判据：闸门偏差 $\leq 1e-06$ pu (实测 **0.000e+00 pu**)；三项守恒残差 ≈ 0 (实测 **0.0000 MW**)；AC 目标值 $>$ DC 目标值且差值可解释。

实测参照 (IEEE 39)：AC 目标 **41,872.30** / 网损 44.18 MW / 最高电压 1.0600；DC 目标 **41,263.94**；差 608.36 (**+1.474%**)；LMP 最高母线 38 = 14.1064。

步骤 13 — 归档与报告

把整个分析过程整理成一份报告，必须包含：

- 1) 算例档案：文件名、格式、母线/支路/机组/负荷数、IR 的 schema 与 version；
- 2) 环境档案：powermcp / powerio / pandapower / pypsa / surge 的版本号；
- 3) 基态结果：收敛状态、电压极值及其母线、最重载线路、守恒残差；
- 4) 交叉校验：两引擎对比的最大偏差、编号偏移探测结果；
- 5) 灵敏度：PTDF/LODF 维度与稀疏度、自治性核对结论；
- 6) N-1 结果：扫描规模、Top-5 关键故障（含断号与数值）、低压故障清单；
- 7) 缓解措施：匹配的手册、措施前后对照表；
- 8) OPF：AC/DC 目标值、LMP 前列母线、达上限机组、守恒校验；
- 9) 诚实声明：本次哪些步骤实测通过、哪些失败或被跳过、哪些结论是推断。

报告中每一条数值结论都要能追溯到具体的工具调用。

如果某一步没做成，就写“未做”并说明原因，不要用推测填充。

验收判据：报告里的每个数字都能对上前面步骤的输出；未完成项被显式标注。

8.4 一次跑完的总提示词

若不想分步交互，可把下面这段整体发出（**但不建议**——分步能及时止损）：

请对 D:\GridData 下的算例做一次完整分析，严格按下面的顺序，每完成一步先向我汇报再继续：

1. powermcp doctor 环境预检（只读，不安装任何东西）
2. 确认 POWERIO_MCP_ALLOWED_ROOTS 是否包含 D:\GridData（只报告，不改系统变量）
3. 列出 GridData 全部文件 + 对 powerio/pandapower/pypsa/surge 各调一次 list_tools 核对工具名
4. 用 powerio parse 解析主算例，保存 IR，报告 schema/version/value_type
5. 对 IR 调 summarize + diagnostics，有 error 就停下问我
6. 导入 pandapower 跑基态潮流，给电压极值/最重载/越限统计 + 守恒校验
7. 用同一份 IR 导入 pypsa 复算，自动探测编号偏移后逐项对比（阈值 $1e-06$ pu）
8. surge 算 PTDF/LODF，做对角线 = -1 的物理自治性核对
9. surge 做 N-1 扫描，给 Top-5 关键故障（断号 + 负载率 + flow）
10. 按 Escalation triggers 表匹配缓解手册，按手册顺序给措施方案（先不改算例）
11. 实施第 1 项措施（用 typed edits），复算并给前后对照表
12. 走 pandapower runopp/rundcopp 做 AC/DC OPF，先过一致性闸门（ $1e-06$ pu）
13. 汇总报告，含环境档案、全部数值、以及“未做/失败/推断”的诚实声明

纪律要求：

- 不许跳过基态潮流直接做故障分析
- 不许在编号未对齐的情况下跨引擎对比
- 不许把推断写成实测结论
- 每一步都要有具体数值和元件编号，不接受模糊表述
- 遇到依赖缺失，报告缺什么、怎么补，不要盲目重试

8.5 本流程的诚实边界

项	说明
流程编排	按已验证能力编排，顺序遵循 PowerSkills 的渐进式披露原则
工具与判定方法	每一步用到的工具、阈值（1e-06 pu）、校验方式（守恒、对角线 = -1）均有实测出处
提示词	未逐条实跑 ——它们是操作模板，不是已验证的对话记录
实测数值	文中的 IEEE 39 数值全部来自实际运行日志，可逐项复核
未覆盖	OpenDSS / ANDES / Egret / LTSpice / 商业 server 的分析流程未纳入（对应 server 本机不可用）

九、已知限制与注意事项

9.1 环境限制

限制	说明	应对
商业许可	PSS/E、PowerWorld、PowerFactory、PSCAD 均需有效许可	先用开源组合验证价值
Windows 绑定	PowerWorld 走 COM 自动化；PSCAD 依赖 <code>mhi-pscad</code> （仅 Windows）	非 Windows 环境选开源方案
Python 版本冲突	surge 需 3.12–3.14；PSS/E 需 numpy 1.x	用独立虚拟环境隔离
PyPSA 版本	需 $\geq 0.35.2$ 且 < 2	注意不要装到 2.x
求解器系统库	surge 需 HiGHS/Ipopt 系统库， <code>pip install highspy</code> 不够	见 4.3 节安装表
外部运行时	HOPE / GenX 需 Julia 运行时与各自仓库本体	装 Julia，或先跳过这两个 server
商业软件版本陷阱	PowerWorld Education/GOS 版不注册 SimAuto COM ，装了 <code>esa</code> 也无法自动化	需商业授权版；先做三级探查确认

9.2 明确的功能边界

文档中坦诚标注的不足：

- **ANDES 服务器**： "under active development and may need further modification"（开发中，可能仍需调整以处理某些内部输出并确保完整兼容）
- **GenX**： "These functionalities are for after a GenX run has been manually configured"（仅适用于手动配置好的 GenX 运行之后）——它不做求解，只做结果后处理
- **PLEXOSDB**： `plexosdb-mcp` 工具层不在 PyPI 上，需按各目录 README 手动安装
- **surge**： 需要 Python 3.12–3.14（pyo3/maturin 的版本范围限制）

- **pandapower server**: 8 个工具中**没有 OPF**，与 PyPSA 能力不对称
- **OpenDSS server**: 55 个工具中**没有任何谐波类工具**

9.3 实践避坑清单

坑	现象	正确做法
安装向导	直接回车只装核心工具	用空格键勾选
ANDES 动态	传了 <code>.dyr</code> 但时域仿真没有动态	检查 <code>dynamic_models_loaded</code> 是否为 True
PSS/E 命令	直接 <code>run_psspy_command</code> 报参数错误	先 search → 再 lookup → 最后 run
特征值参与因子	用列表位置索引结果不对	用 mode 的 <code>index</code> 字段
surge 求解器	只 <code>pip install highspy</code> 后仍失败	装系统库 <code>libhighs-dev / coinor-libipopt-dev</code>
Codex 配置	Desktop 应用后配置被重置	重新运行 <code>powermcp install</code>
PowerIO 路径	非本地 URI 被拒绝	只用本地路径或 <code>file://</code>
多状态包	不指定运行点直接报错	显式传 <code>scenario_id / time_index</code> 或 <code>study_commit</code>
工具名	调用报"找不到工具"	以 <code>list_tools</code> 实际返回为准 (OpenDSS 已有 6 个失效名先例)
读文件被拒	报路径不在允许根内	配置 <code>POWERIO_MCP_ALLOWED_ROOTS</code> ，这是策略不是 bug
配电网算例	被平衡求解器拒绝	调 <code>to_balanced_report / to_balanced</code> 降维

9.4 失效模式对照排查表

流程中断时先查这张表——**每一种都是实测到的真实失效模式**：

症状	真实成因	处置
<code>'pandapowerNet' object has no attribute 'deepcopy'</code>	pandapower 3.5.4 已移除该方法；上游未适配	换用 surge 做 N-1（输出更完整）；或应用修复
脚本打印"未收敛"，但求解器返回 <code>('ok','optimal')</code>	PyPSA 已移除 <code>Network.status</code> ， <code>hasattr</code> 为假 → 误报失败	改从 <code>optimize()</code> 返回值取状态； 不要据此放弃可行路径
传了 <code>linearized=True</code> 但结果像 AC OPF	该参数不存在，实参落进 <code>**kwargs</code> 被 静默忽略	关键参数先核对函数签名
跨引擎对比出现 0.0x pu 的"差异"	两侧编号约定不同，字面键对比拿错元件	自动探测编号偏移，不要硬编码

症状	真实成因	处置
No LP solver found – install HiGHS	surge 需系统 C 库，pip install highspy 不提供共享库	设 HIGHS_LIB_DIR 指向含 highs.dll 的目录
No AC-OPF NLP solver found	surge 的 AC OPF 需 Ipopt 或 Gurobi 13	AC OPF 改走 pandapower runopp
Gurobi 13 not found	版本不匹配（本机 11.0.3）+ 许可过期	依赖 Gurobi 的路径不可用，改用内建求解器
工具调用报"找不到工具"	工具名不存在	以 list_tools 实际返回为准
工具调用报缺 Julia / COM	HOPE/GenX 缺 Julia；PowerWorld 版本不含 SimAuto COM	补运行时，或换商业授权版

十、能力边界：PowerMCP 不做什么

理解边界和理解能力同样重要：

PowerMCP 是	PowerMCP 不是
工具接入层——让 LLM 能调用软件	X 决策层——不会自己判断该做什么分析
暴露各软件的原始能力	X 不替代专业的电力分析软件
提供标准化的调用协议	X 不做工作流编排（那是 PowerWF 的事）
提供跨格式翻译（PowerIO）	X 不做领域知识与处置经验（那是 PowerSkills 的事）

关键认知：PowerMCP 给了智能体"手"，但没有给"经验"。一个拥有全部工具的智能体，可能一上手就跑 N-2 分析，而实际上基态潮流都还没收敛——这是初级工程师的典型错误。

解决这个问题的是 **PowerSkills**（技能层，教智能体"先做什么、验证什么、何时升级"）和 **PowerWF**（编排层，把工具串成业务流程）。三者配合才构成完整的解决方案。

建议的搭配顺序：PowerMCP（能调用）→ PowerSkills（知道怎么做）→ PowerWF（成流程）。

十一、PowerSkills：补上"经验"的那一层

第十章说明了 PowerMCP 的边界：它给了"手"，没给"经验"。PowerSkills 就是补这一层。

11.1 结构与定位

PowerSkills 是 PowerMCP 之上的**技能层**，回答三个问题：**先用哪个工具 → 升级前验证什么 → 发现真问题时按什么手册处理。**

PowerSkills/	
└─ powerskills-tool/skills/	11 个软件工作流技能
│ └─ pandapower pypsa surge potpourri	← 有对应 PowerMCP server
│ └─ andes egret ltspice opendss	
│ └─ powerworld pslf psse	← 对应 server 未装/需授权
└─ powerskills-engineering/skills/	10 个缓解手册 (playbook)
└─ skill-creator/	技能编写与校验工具 (不属于任一插件)

渐进式披露 (progressive disclosure) 是 tool skill 的核心设计：强制顺序为**载入算例 → 检查模型 → 求解基态 → 修改 → 高级研究**，避免直接跳到故障分析、动态仿真或投资优化。这正是第八章那条 13 步流程的设计依据。

11.2 11 个软件工作流技能

Skill	覆盖范围	先暴露的工具	后续升级到的工具
surge	输电分析、灵敏度、OPF、故障、ATC、调度	load_builitn_case / load_network 、 get_network_info 、 run_ac_power_flow	compute_ptdf / compute_lodf 、 run_dc_opf / run_scopf 、 run_n1_branch_contingency 、 compute_nerc_atc 、 run_scuc
pandapower	AC 分析与筛选研究	load_network / create_empty_network 、 get_network_info 、 run_power_flow	run_contingency_analysis
PyPSA	规划、OPF、扩建研究	load_network / create_network 、网络检查	optimize_network 、 optimize_investment 、导入导出
potpourri	pandapower 配网的 AC/DC 与多时段 OPF (含柔性资源与储能)	scripts/inspect_case.py 、 scripts/solve_opf.py --list-solvers	--formulation dc → -- formulation ac 、 --horizon N - -battery-penetration
ANDES	动态安全与小信号研究	run_power_flow 、 get_system_info	run_eigenvalue_analysis 、 run_time_domain_simulation
Egret	市场与运行优化	solve_dc_opf	solve_ac_opf 、 solve_unit_commitment_problem
OpenDSS	配电网馈线研究	compile_opendss_file 、 solve_snapshot	负荷倍率、日能量表、谐波 (⚠ 见 11.5 节)
PSLF	输电潮流与故障	open_case 、 solve_case 、越限检查	拓扑/设备编辑、 run_contingency_analysis
PSSE	PSS/E 基态与 API 引导研究	open_case 、 solve_case	lookup_psspy_command 、 search_psspy_commands 、 run_psspy_command
PowerWorld	稳态分析与灵敏度	open_case 、 run_powerflow 、结果查询	故障、参数修改、PTDF/LODF/雅可比

Skill	覆盖范围	先暴露的工具	后续升级到的工具
LTSpice	电路仿真流程	<code>create_simulation_session</code> 、 <code>run_simulation</code>	<code>list_available_traces</code> 、 <code>plot_specific_traces</code> 、GUI

`potpourri` 是唯一**没有** PowerMCP server 的技能，完全靠技能自带 Python 脚本驱动。

11.3 10 个资深工程师缓解手册

每个手册都是"从观测到的问题出发 → 按资深工程师真实会尝试的顺序列出纠正措施"：

Skill	适用时机	典型动作
voltage-violation-mitigation	低压/高压、无功支撑不足、分接头用尽	AVR / Volt-VAR 检查、并联补偿、分接头、再调度、减少输电
thermal-overload-mitigation	线路或变压器过载	再调度、拓扑切换、移相器、加固筛选
contingency-mitigation	N-1 / N-2 越限或校正措施薄弱	故障前修正、校正切换、RAS 筛选、长期升级
dynamic-stability-mitigation	阻尼差、暂态失稳、电压恢复慢	模型检查、出力缓解、AVR/PSS/调速器整定、动态无功
operations-planning-mitigation	OPF/UC 不可行、高弃电、阻塞、备用不足	数据清洗、简化筛选求解、增加灵活性、约束复核
convergence-failure-mitigation	任何工具潮流发散/不收敛	数据检查、孤岛与平衡机复核、分阶段放宽控制、降低压力
short-circuit-mitigation	短路容量超断路器额定	研究复核、母联分裂、串联电抗器、断路器升级
frequency-response-mitigation	低惯量、频率最低点差、RoCoF 高、UFLS 风险	调速器裕度、快速频率响应、增加惯量、下垂/死区修正
interconnection-impact-mitigation	新增发电/储能/大负荷并网筛查	POI 处 N-1 筛查、SCR/弱网检查、无功要求、升级定容
der-hosting-capacity-mitigation	分布式电源引起电压抬升、反向潮流、保护灵敏度下降	逆变器 Volt-VAR/出力限制、调压器整定、保护复核、加固

11.4 跨层契约：Escalation triggers

每个 tool skill 末尾都有一张 **Escalation triggers** 表，把**具体观测值**映射到对应的缓解手册——交接由**数字**驱动而非感觉：

tool skill 触发行	目标 playbook	对齐
<code>res_bus.vm_pu < 0.95 or > 1.05</code>	voltage-violation-mitigation	✓
<code>loading_percent > 100</code>	thermal-overload-mitigation	✓

tool skill 触发行	目标 playbook	对齐
<code>run_n1_*</code> 有 binding contingency	contingency-mitigation	✓
潮流不收敛	convergence-failure-mitigation	✓

交叉一致性实测：引用的 mitigation skill 10 个 ↔ 实际存在的 10 个，**无悬空引用、无孤儿 skill**。这是本仓库设计中最扎实的部分。

11.5 技能健康度审计

对上游仓库的审计结论：**21 个 skill 中，4 个在役 tool skill 里有 3 个存在阻断级问题**；surge 是唯一质量完好的。

严重度	数量	代表问题
阻断	3	<code>optimization_analysis.py</code> 语法错误 （markdown 围栏当成了 docstring 终止符，文件无法导入）；pandapower 的 <code>net.deepcopy()</code> ；pypsa 的 <code>Network.status</code> （ 求解成功却误报"未收敛" ）
事实不符	4	pypsa 自带算例 潮流失效 （无法复现声称的 87%）；线路/变压器数量写反（34/12 → 实为 35/11）；loads 数不符（19 → 实为 21）；脚本与自带算例命名不兼容
引用错误	1	<code>opendss/SKILL.md</code> + README 引用的 6 个工具名全部不存在
规范/结构	5	Itspice 缺 escalation 表； 仓库无 CI ；版本约束前后不一致
通过	3	escalation 引用完整（10/10）· 工具名有效（除 OpenDSS）· 触发条件对齐

根因共性：代码写于旧版 API + 依赖无版本上界 + **仓库无 CI** → 缺陷长期存活。自检脚本其实存在且确实能抓到 `deepcopy` 缺陷，但**没有任何东西自动运行它**。

特别提示（比崩溃更隐蔽）：pypsa 的 `Network.status` 问题是**误报失败**——求解器明确返回了 `('ok','optimal')`、目标值 168,246.9，脚本却打印"Baseline optimization did not converge!"。这比崩溃更危险，因为 Agent 会据此**主动放弃一条其实可行的路径**。

审计范围局限（诚实声明）：本次只覆盖 4 个在役 tool skill 的动态验证 + 跨层交叉一致性检查；其余 17 个 skill 的静态文档审计**未做**。potpourri / surge-OPF / OpenDSS 因环境依赖缺失未做动态验证（报告中已区分"skill 缺陷"与"环境限制"）。其中 pandapower / pypsa 的 4 处同源 API 漂移缺陷已修复并提交上游 PR（3 文件 +39/-12）。第 4 处是**验证驱动**发现的——修完前 3 处重跑，错误才从"误报未收敛"推进到 `mremove`。**只看代码不跑，PR 会是不完整的。**

十二、科研应用场景示例

标注含义：**已具备**=有实测素材可直接开工；**需补环境**=缺依赖；**需构造**=需先准备数据集。

12.1 场景清单

#	场景	研究问题	工具链	可行性
1	N-1 扫描与关键故障排序	如何在可复现流程下得到带数值的 N-1 关键故障排序，并验证其物理自治性？	surge run_n1_branch_contingency	已具备
2	PTDF/LODF 灵敏度与阻塞归因	如何用灵敏度矩阵解释特定线路的阻塞成因与相互影响？	surge compute_ptdf / compute_lodf	已具备
3	AC/DC OPF 对比与网损影响量化	AC 与 DC 的目标值差异有多少来自网损？LMP 空间分布如何？	pandapower runopp / rundcopp	已具备
4	跨求解器一致性与转换保真度	同一算例在不同引擎间转换后，结果能保持一致到什么精度？	powerio parse / emit / calc_matrix	已具备
5	潮流不收敛的成因诊断	同一份数据，为何某引擎可解而另一引擎判不可行？	pandapower + PyPSA 对照	已具备
6	求解器容错行为对比	面对不自洽数据，不同引擎的容错策略如何？	pandapower + PyPSA 对照	已具备
7	LLM 调用电力工具链的可靠性	LLM 驱动专业电力工具时，失效模式有哪些？如何量化？	全链路（失效模式分类）	已具备
8	批量算例扫描与数据集构建	如何扩展为跨算例、跨负荷水平的数据集？	surge N-1 + 矩阵 + OPF 批量循环	需构造
9	再调度 / RAS 措施量化评估	给定 N-1 越限，各类缓解措施的效果与代价如何量化？	surge N-1 + engineering 手册 + edits	需构造
10	配电网分布式电源承载力	馈线可接入多少分布式电源而不引起电压越限？	OpenDSS 或 potpourri 多时段 OPF	需补环境

12.2 几个场景的展开

场景 3：AC / DC OPF 对比与网损影响量化

实测产出可直接作为论文素材：AC 目标 **41,872.30** vs DC **41,263.94**，差 608.36（**+1.474%**）——这个差值正是 AC 计及 44.18 MW 网损的结果。同时可给出节点边际电价的空间分布（LMP 最高母线 38 = 14.1064，前 5 高集中在母线 38/8/0/7/6）。**校验手段**：三项守恒校验残差均为 0.0000 MW。

场景 4：跨求解器一致性与转换保真度

实测产出：同一份 PowerIO IR 经 pandapower 与 PyPSA 两个独立求解器，母线电压最大偏差 **6.98e-11 pu**、支路有功 **2.25e-07 MW**——保真度达到机器精度。**使用陷阱**：两侧编号约定不同（0-based vs 1-based），必须自动探测偏移，否则会拿不同物理元件对比（首次曾误报 0.068 pu）。

场景 5：潮流不收敛的成因诊断（真实研究素材）

素材：PyPSA 自带 `case39.nc` 潮流**发散**（p0 达 1e38 量级），而 `optimize()` 返回 `('ok','optimal')` ——**LOPF 可解、PF 不可解**。

已做的排除（6 项，均为实测）：`pf()` 本身可用（极简 2 节点网络正常）、网络连通（1 个连通分量）、无零阻抗支路、容量无缺失、LOPF 可解、手动设 `control='Slack'` 仍不收敛。

已定位但不充分：`build_case39.py` 中的 slack 机组从未设置 `control="Slack"`，网络无参考母线。

诚实边界：根因**未完全定位**——实测把该机组手动设为 Slack 后潮流仍发散，故它不是充分原因。这是一份"根因未定位"的真实素材，适合作"求解器鲁棒性与数据自治性"的研究起点。**不得把"根因未定位"写成"已定位"**。

场景 7：LLM 调用电力工具链的失效模式研究

这是本项目实测中最有独特价值的研究点。已有 5 类可量化的失效模式（全部有实测记录）：

- 1. **误报失败**——`Network.status` 不存在 → `hasattr` 为假 → 打印"未收敛"，而求解器实际返回 `('ok','optimal')`
- 2. **静默忽略参数**——不存在的 `linearized` 落进 `**kwargs`，跑的一直是 AC OPF，"DC 路径从未被真正测试"
- 3. **工具名冲突**——多 server 同名工具无法靠名字消歧
- 4. **无降级路径**——依赖缺失只有调用才失败，无 mock 降级
- 5. **编号约定不一致**——字面键对比拿错物理元件（首次误报 0.068 pu）

产出：可量化的失效模式分类 + 对应的缓解设计（一致性闸门、可用性预检、显式 server 指定、编号偏移探测）。

十三、工程应用场景示例

#	场景	业务问题	流程与工具链	可行性
1	规划算例自动体检	拿到一份新算例，如何快速判断它"健康"还是"有问题"？	载入 → 基态潮流 → 电压/过载统计 → N-1 → 报告	已具备
2	商业软件自动化可行性预检	本机装了商业软件，是否就能自动化调用？	三级探查：软件是否安装 → COM/授权是否注册 → 桥接包是否就位	已具备
3	算例格式批量迁移	历史算例散落在多种格式，如何统一？	<code>powerio parse</code> → IR → <code>emit(format=...)</code>	已具备
4	检修方式编排与 N-1 校验	某线路检修期间，系统是否仍满足 N-1？	PowerIO typed edits 写入检修状态 → 导入求解器 → N-1 扫描	已具备
5	运行成本优化与阻塞定价	当前运行方式下最小成本是多少？哪些母线电价最高？	AC OPF（ <code>runopp</code> ）→ 目标值 + 机组出力 + LMP	已具备
6	越限问题的标准化缓解	N-1 发现线路过载，下一步做什么？	越限观测 → Escalation triggers 匹配 playbook → 按手册顺序尝试	已具备

#	场景	业务问题	流程与工具链	可行性
7	并网影响筛查	新增电源/负荷接入某节点，POI 处是否仍满足 N-1?	interconnection-impact 手册 → POI N-1 筛查 → SCR 检查 → 无功核算	已具备
8	送电能力（ATC）评估	指定断面可增加多少输电能力?	surge <code>compute_nerc_atc</code>	需验证
9	配电网分布式电源接入评估	台区/馈线电压抬升、反向潮流如何处理?	OpenDSS 或 potpourri → der-hosting-capacity 手册	需补环境
10	短路容量与断路器校核	故障电流是否超断路器额定?	short-circuit-mitigation 手册（措施清单）	需商业授权

13.1 几个场景的展开

场景 1：规划算例自动体检与体检报告

实测产出模板（IEEE 39）：收敛 ✓ / 最低电压 0.9820（母线 30） / 最高 1.0636（母线 35） / 最重载 73.37%（线路 21） / 过载 0 条 / N-1 越限 23 个 / 网损 43.6411 MW / 守恒残差 0.0000 MW。

工程价值：把"人工点开软件逐个看"变成**可批量、可留痕、可版本对比**的流程。

报告措辞纪律：母线 35 的 1.0636 **恰等于该处机组电压设定值**（PV 母线维持设定点），反映的是数据设定偏高而非求解异常——报告必须区分这两者。

场景 2：商业软件资产盘点与自动化可行性预检

答案：软件装了不等于能自动化。实测案例：PowerWorld 已安装，但为 `Simulator G0S Education 23` 版，**不注册 SimAuto COM 类**，即便装上 `esa` 也无法工作。探查输出为 `[SKIP] PowerWorld COM/License unavailable`，缺失项：SimAuto COM 注册，`esa` 包。

工程价值：避免在不可自动化的资产上浪费排期——这类"看起来能跑、实际跑不动"的坑，只有在真正调用时才会暴露。

场景 3：算例格式批量迁移

实测保真：跨引擎 $\Delta = 6.98e-11$ pu，**转换不引入可观测误差**。

前提注意：`Package` / `model-json` / `package_json` / `study_commit` 为已退役格式，需重新 parse 原算例。**已知限制：**IR → PyPSA 导入**不携带** `q_min_pu` / `q_max_pu`（无功能力限值）。

场景 4：检修方式编排与 N-1 校验

用 PowerIO 的 **typed edits** 把检修状态写成结构化变更（`set_branch_in_service` / `set_switch_closed` / `set_transformer_tap_ratio` 等 12 类 op）→ 导入求解器 → N-1 扫描。

工程价值：检修方案**先验后执行**，且变更**可审计**——返回的 `edits` 报告按应用顺序列出被改动的元件。

场景 6：越限问题的标准化缓解流程

实测映射：`loading_percent > 100` → `thermal-overload-mitigation`；`vm_pu < 0.95` → `voltage-violation-mitigation`。

工程价值：把"资深工程师的经验顺序"固化成**可执行、可复用的** SOP，降低对个人经验的依赖。**边界：**手册给出的是措施顺序，具体效果仍需重算验证。

13.2 场景落地优先级

优先级	场景	理由
P0（立即可做）	科研 1/2/3/4；工程 1/2/3/5	全部有现成素材 + 实测数值 + 校验手段， 零前置依赖
P1（小成本）	科研 5/6/7；工程 4/6/7	有素材，需整理为正式研究/流程文档
P2（需决策）	科研 8/9/10；工程 8/9/10	需先构造数据集，或先决策环境变更（补装 extra / 商业授权）

落地前的前置条件检查清单：

- 1. `powermcp doctor` — 确认目标 server 依赖 `ok`
- 2. 依赖预检（场景 2 模式）— 确认外部运行时/COM/授权就位
- 3. 求解器可用性 — surge 的 DC OPF 需 `HIGHS_LIB_DIR`（当前未持久化，仅在 shell 会话内）
- 4. 版本核对 — 目标 skill 的 requirements 是否与在役版本冲突
- 5. 工具名核对 — 确认工具真实存在于目标 server（尤其 OpenDSS）
- 6. 一致性闸门 — 凡走 Python 侧路径，必须验证网络与 MCP 侧为同一个

十四、快速上手清单

```
# ① 最小可用安装
pip install powermcp
powermcp install      # 空格勾选工具
powermcp doctor       # 验证

# ② 在 MCP 客户端里确认服务器已连接
#   Claude Desktop / Claude Code / Cursor 都应能看到 pandapower、pypsa、powerio

# ③ 第一个测试对话
#   "加载 case39.json，跑潮流，报告母线电压范围和最重载线路"
```

建议的第一个完整练习：用第七章示例 1（IEEE 39 节点 N-1 分析）走通全流程。它同时验证了案例加载、潮流求解、故障分析三条链路，而且输出结果有公开基准可以对照，方便判断是否配置正确。

扩展方向：

- 想研究规划优化 → 加 PyPSA，试 `optimize_network` / `optimize_investment`
- 想做动态分析 → 加 ANDES，注意先确认 `dynamic_models_loaded`
- 想覆盖全流程 → 加 surge（注意系统库 HiGHS）
- 想跑通完整分析流程 → 按第八章的 13 步走一遍
- 有商业软件 → 按 3.4 节配置路径，用 `powermcp doctor` 验证，**并先做三级可行性探查**

最后一条建议：无论用哪个 server，都请记住 6.5 节的核心结论——**装得上 ≠ 跑得动**。开工前花两分钟做可用性预检，比在流程中途遇到依赖缺失要划算得多。

附录 A：资源索引

资源	链接
仓库主页	https://github.com/Power-Agent/PowerMCP
PowerSkills（技能层）	https://github.com/Power-Agent/PowerSkills
MCP 协议文档	https://modelcontextprotocol.io/introduction
MCP Python SDK	https://github.com/modelcontextprotocol/python-sdk
PowerIO	https://github.com/eigenergy/powerio
Easy SimAuto (PowerWorld 桥)	https://github.com/mzy2240/ESA
ANDES 文档	https://andes.readthedocs.io/
PyPSA	https://pypsa.org/
pandapower	https://github.com/e2n1EE/pandapower
故障评估演示视频	https://www.youtube.com/watch?v=MbF-SlBI4Ws
负荷增长评估演示视频	https://www.youtube.com/watch?v=euFUvhhV5dM
官方教程 PDF	仓库根目录 PowerMCP_Tutorial1.pdf （注意：这是旧版手动配置方式，非推荐路径）

核心团队：Qian Zhang、Steven Black、Paulo Radatz、Andrea Pomarico、Muhy Eddin Za'ter、Luan Lopes dos Santos、Stephen Jenkins、Maanas Goel、Shen Wang、Drew Gray、Samuel Talkington

维护方：哈佛大学 SEAS 电力与人工智能倡议（Power and AI Initiative, PAI）

附录 B：与第一版的差异、依据与可追溯性

B.1 本版相对第一版的更正

第一版（2026-09-19）基于仓库源码与官方文档整理。本版并入了一轮完整实测的结论，因此更正了以下内容：

项	第一版	本版（实测）
服务器 / 软件数量	15	16 （本仓 15 + PowerIO 由依赖提供）
case39 算例规模	34 线路 / 12 变压器	35 线路 / 11 变压器 ；9 机组 + 1 平衡机 / 21 负荷
示例 1 输出	L17 = 105.5%、母线 7 = 0.9473 pu	母线 30 = 0.9820 pu；线路 21 = 73.37%；N-1 Top-1 branch_26 = 161.84%

项	第一版	本版（实测）
OpenDSS 工具名	<code>compile_and_solve</code> 等 6 个	该 6 个名称全部不存在；已更正为 <code>compile_opendss_file</code> + <code>solve_snapshot</code> 等（见 4.7 节）
PowerIO 工具名	<code>matrix(kind=...)</code> 、 <code>save(to_format=...)</code>	<code>calc_matrix(matrix=...)</code> 、 <code>emit(format=..., destination=...)</code> （0.11.x 现行 API）
PyPSA DC OPF	（第一版无此结论）	曾出现的"DC OPF 亦 infeasible"结论 作废 —— <code>linearized</code> 参数不存在，被静默忽略，实际跑的一直是 AC OPF
排版错字	"Powerlo"	PowerIO

B.2 本文结论的证据来源

结论	证据来源
16 server / 约 250 工具 / 工具名冲突	仓库扫描报告（提交 <code>a21ea6b</code> ，v0.4.0）
运行时 <code>list_tools</code> 核验（8/17/44/20/10）	环境搭建记录；核验脚本实际拉起 server 调 <code>list_tools</code>
IEEE 39 基态 / N-1 / PTDF / LODF 数值	示例脚本运行日志
IR 保真 $\Delta = 6.98e-11$ pu	跨格式转换示例日志
AC/DC OPF 与 LMP	OPF 示例日志
商业 server 探查结论	商业/高依赖 server 探查日志
pandapower <code>deepcopy</code> 缺陷与修复	两条独立路径互证：MCP 直调与注入补丁均为 46 故障 / 45 越限
PyPSA OPF infeasible（13 项排除）	验证报告（含"零负荷仍不可行"的退化案例）
surge HiGHS 可用 / Ipopt 未解决 / Gurobi 阻塞	求解器配置记录（HiGHS 连续 3 次一致）
21 skill 健康度（3/4/1/5/3）	PowerSkills 审计报告
PR 修复 4 处	审计报告；4 处均为实测试验

B.3 本版明确未做的事

- 未新增任何实验或数据——全部数值均转引自实际运行日志
- 未验证 OpenDSS / ANDES / Egret / LTSpice / PSSE / PSLF / PowerFactory / PSCAD / PLEXOSDB / PowerWorld / HOPE / GenX 的**功能**（仅核验了工具清单）
- 未做其余 17 个 skill 的静态文档审计
- 未定位 PyPSA OPF 在 IR 导入算例上不可行的根因
- 第八章的提示词**未逐条实跑**——它们是操作模板，不是已验证的对话记录
- 第十二、十三章的场景除标注"已具备"者外，**均未实际执行**

- 第九章 4.3 节标注的"surge Windows Iptopt 不可用"是基于"偶发成功 + 静默崩溃 + 二进制字符串分布"的**推测**，未做进一步验证，不作为上游缺陷断言

本文档基于 PowerMCP 仓库源码、官方文档，以及一轮完整的本机安装与实测（Python 3.12.6 隔离环境 • powermcp 0.4.0 • powerio 0.11.3 • pandapower 3.5.4 • pypsa 1.2.4 • surge-py 0.1.9）整理。

初版生成时间：2026 年 9 月 19 日 | 本版（并入实测结论）：2026 年 9 月 21 日